EXPERIMENT 9

Aayush Patil
D20A-56

AIM : To create an ERC 20 token in Remix IDE and deploy it to a testnet using a MetaMask wallet.

THEORY :

ERC 20 token - A fungible blockchain token built on a common interface for token balances, transfers and allowances. Every token unit is identical and interchangeable. ERC 721, in contrast, is meant for non-fungible tokens whose assets are individually distinguishable.

MyToken contract - It defines the public attributes name, symbol, decimals and totalSupply, together with the balanceOf and allowance mappings. The constructor creates the initial supply and assigns it to the deploying account.

Functions - transfer sends tokens, approve grants a spender an allowance, and transferFrom moves tokens using an approved allowance.

Events - The contract emits Transfer and Approval events whenever token balances or spending permissions change.

Remix and MetaMask – Remix IDE is used to compile and deploy the contract, while MetaMask approves the transactions on the selected testnet.

CODE AND OUTPUT

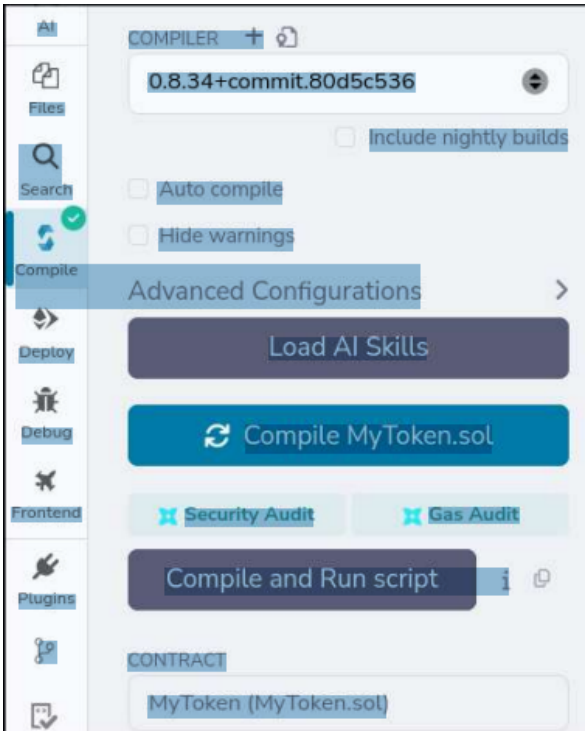
Step 1 - Write the MyToken.sol contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract MyToken {
=
string public name string public symbol uint8
public decimals
"Jai Token";
= "JAI";
=
18;
uint256 public totalSupply;
mapping (address => uint256) public
balance0f;
mapping (address => mapping (address =>
uint256)) public allowance;
event Transfer (address indexed from, address
indexed to, uint256 value);
event Approval (address indexed owner,
address indexed spender, uint256 value);
constructor (uint256 initialSupply) {
totalSupply
=
initialSupply * 10 ** decimals;
balanceOf [msg.sender]
```

```
=
totalSupply;
emit Transfer (address (0), msg.sender,
totalSupply);
function transfer (address to, uint256 amount)
public returns (bool) {
require (balanceOf [msg.sender] >= amount,
"Insufficient balance");
balanceOf [msg.sender]
==
amount;
balanceOf[to] += amount;
emit Transfer (msg.sender, to, amount);
return true;
}
function approve (address spender, uint256
amount) public returns (bool) {
allowance [msg.sender] [spender]
=
amount;
emit Approval (msg.sender, spender, amount);
return true;
}
function transferFrom (address from, address
to, uint256 amount) public returns (bool)
require (balanceOf [from] >= amount,
"Insufficient balance");
require (allowance [from] [msg.sender] >=
amount, "Allowance exceeded");
balanceOf [from]
==
amount;
balanceOf [to] += amount;
allowance [from] [msg.sender]
==
amount;
emit Transfer (from, to, amount);
return true;
}
}
```

Step 2- Compile MyToken.sol

In the Solidity compiler panel, version 0.8.34 is selected and MyToken.sol is compiled. The contract MyToken (MyToken.sol) is listed.

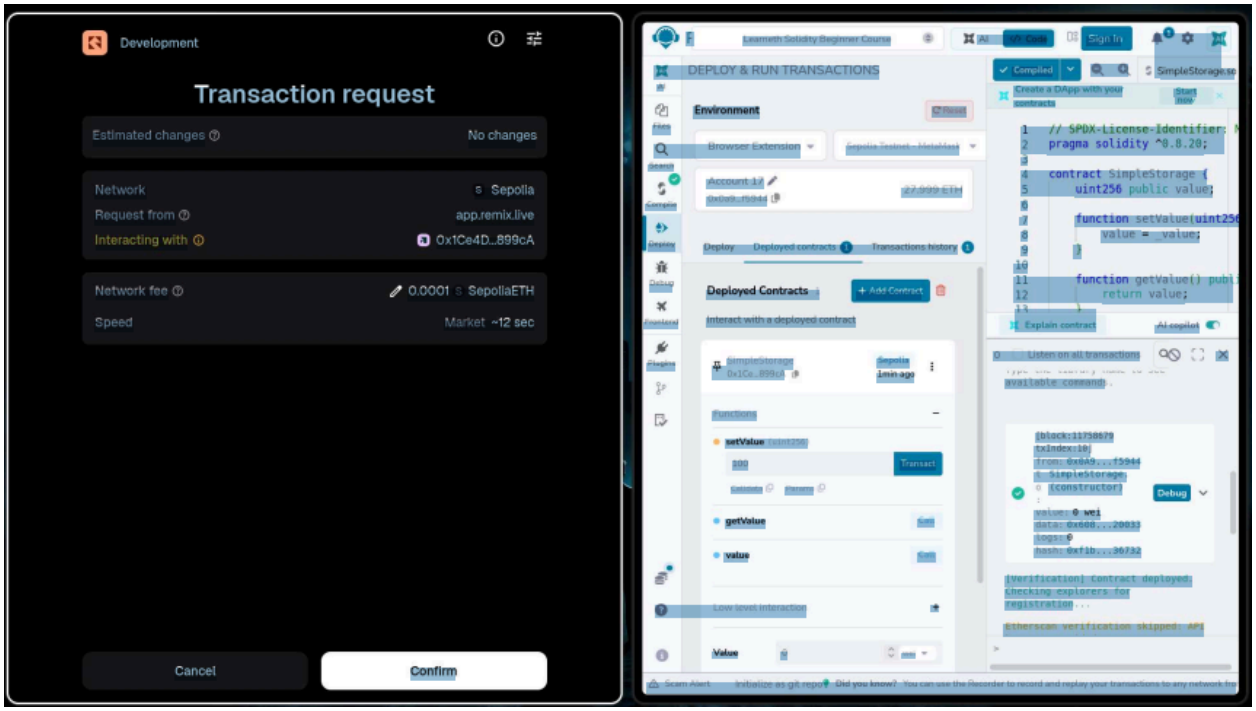


Step 3 Test the MetaMask connection with a SimpleStorage contract on Sepolia

Before deploying the token, a SimpleStorage contract is deployed on Sepolia using the "Browser Extension - Sepolia Testnet - MetaMask" environment. Remix shows the deployment and Sourcify verification as successful.

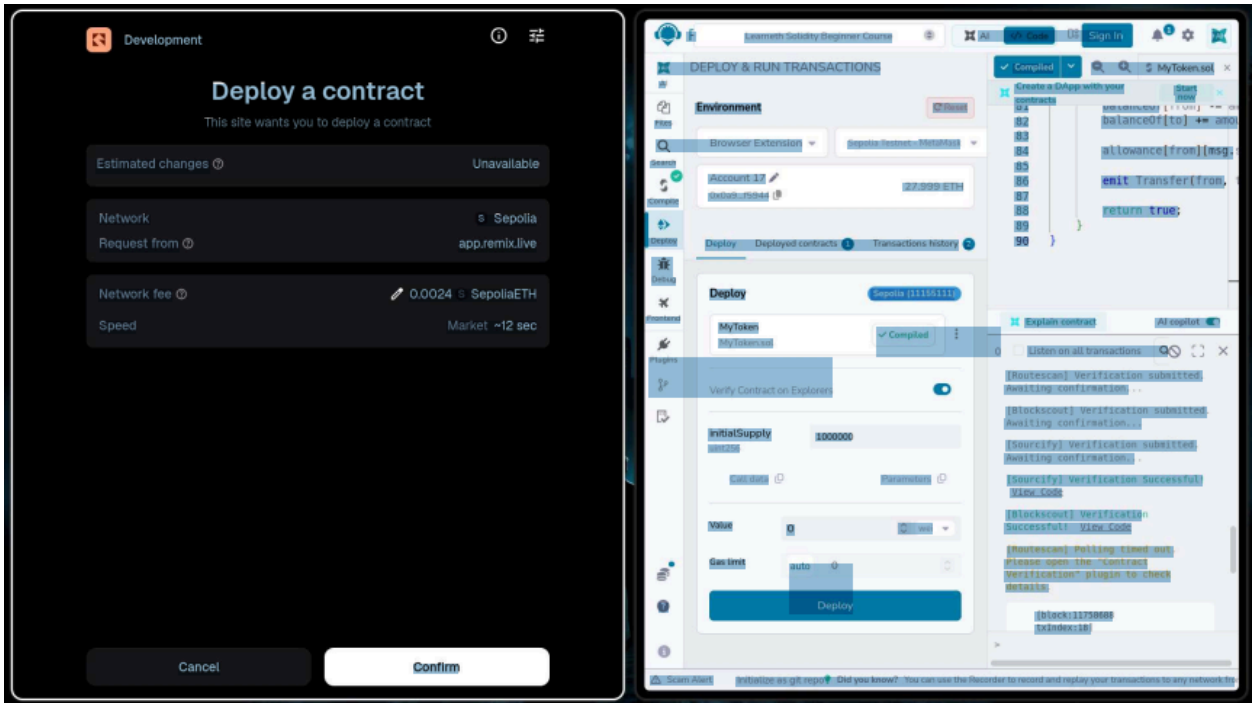


The MetaMask transaction request shows the Sepolia network, app.remix.live as the requester and the network fee.



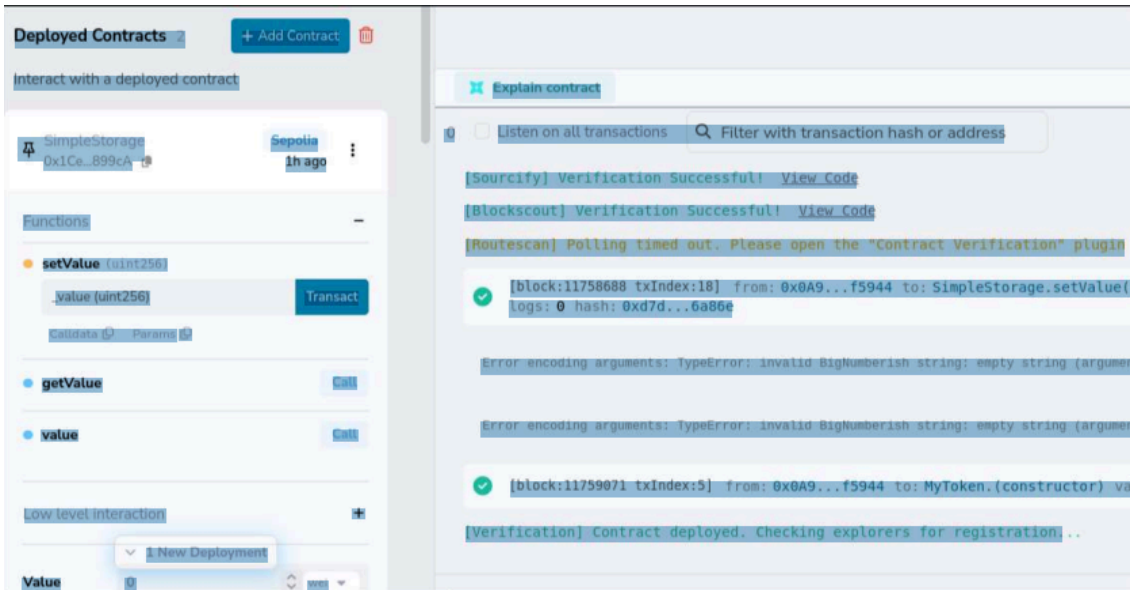
Step 4 - Deploy MyToken and confirm in MetaMask

MyToken is selected in the Deploy panel with initialSupply = 1000000. MetaMask shows a "Deploy a contract" request on Sepolia, which is confirmed.



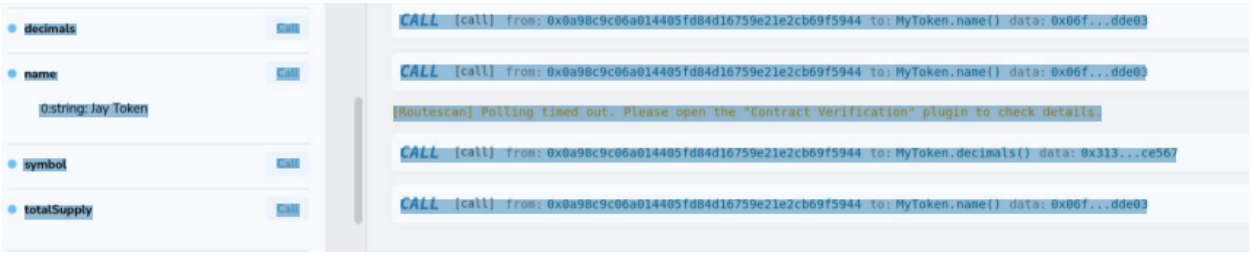
Step 5 - Check the deployed contracts

Deployed Contracts now lists two contracts, and the console shows the successful MyToken constructor transaction.



Step 6 - Inspect the token attributes

The public functions decimals, name, symbol and totalSupply are called. name returns "Jai Token".



CONCLUSION:

The **MyToken** contract was created with ERC 20 token attributes, balance and allowance mappings, **Transfer and Approval events** and the transfer, approve and transferFrom functions. It was compiled in Remix IDE and deployed to the **Sepolia testnet** through MetaMask, and its public attributes were verified. The experiment demonstrated **creating and deploying a fungible token contract** using Remix IDE and a MetaMask-connected testnet.